

Создание клиент-серверного Web приложения при помощи React и Node.js

Оглавление

1. Назначение методического указания	4
2. Необходимые программы и предварительная подготовка.....	5
2.1. Работа с PowerShell и командой cd.....	5
2.2. Два терминала для одного клиент-серверного приложения	7
3. Клиент-серверная архитектура и технологический стек.....	8
3.1. Клиент-серверная архитектура	8
3.2. Роль клиентской части на React.....	9
3.3. Роль серверной части на Node.js.....	9
3.4. Технологический стек проекта	10
3.5. Как клиент и сервер обмениваются данными	10
3.6. Зачем нужен Express	11
4. Установка и первичная проверка Node.js	13
4.1. Проверка установки	14
4.2. Запуск и использование	14
4.3. Выполнение файла	15
5. Создание простого HTTP-сервера	16
5.1. Запуск сервера	16
5.2. Объект request.....	17
5.3. Объект response	18
5.4. Создание рабочего сервера для React-клиента.....	19
5.4.1. Переход от http к Express	19
5.4.2. Структура проекта	20
5.4.3. Создание серверной части	20
5.4.4. Создание файла app.js	21
5.4.5. Разбор кода сервера.....	22
5.4.6. Настройка команды запуска сервера	23
5.4.7. Проверка работы сервера.....	24
6. Создание клиентской части на React.....	26
6.1. Назначение клиентской части	26
6.2. Создание React-проекта	27
6.3. Запуск React-приложения	28

6.4.	Структура клиентского проекта.....	29
6.5.	Точка входа React-приложения.....	30
6.6.	Очистка стартового шаблона.....	31
6.7.	Настройка адреса сервера.....	32
6.8.	Получение данных с сервера через fetch.....	33
6.9.	Разбор кода получения данных.....	35
6.10.	Добавление стилей клиентской части.....	36
6.11.	Проверка работы React-клиента.....	37
6.12.	Передача данных с React-клиента на сервер.....	38
6.13.	Маршрут POST на сервере.....	38
6.14.	Форма отправки данных в React.....	39
7.	Типичные ошибки и способы исправления.....	41
8.	Рекомендуемые источники и справочные материалы.....	43

1. Назначение методического указания

Данное методическое указание предназначено для первого знакомства с Node.js и созданием клиент-серверного веб-приложения.

В качестве клиентской части рассматривается приложение на React. В качестве серверной части используется Node.js. На начальном этапе сервер создаётся на встроенном модуле http, чтобы показать базовый принцип работы запросов и ответов. Далее используется Express, так как он ближе к реальной практике разработки учебных и прикладных веб-приложений.

После выполнения методического указания должен быть получен минимальный, но полностью рабочий пример клиент-серверного приложения: React-клиент отправляет запрос на сервер Node.js, сервер обрабатывает его и возвращает JSON-ответ, который отображается на странице.

В данных методических указаниях рассматривается разработка на операционных системах Windows 10/11 и для выполнения работы потребуются: установленный Node.js, редактор кода, терминал или PowerShell, любой современный браузер и базовые знания JavaScript.

2. Необходимые программы и предварительная подготовка

Перед началом практической работы необходимо подготовить минимальный набор программ. Необходимо установить Node.js, редактор кода и любой современный браузер. Такой набор позволяет создать серверную часть, клиентскую часть и проверить их взаимодействие на локальном компьютере.

Программа		Назначение
Node.js		Среда выполнения JavaScript вне браузера. Используется для запуска серверного кода, npm, Vite и инструментов разработки.
npm		Менеджер пакетов, который устанавливается вместе с Node.js. Нужен для установки зависимостей и запуска команд проекта.
Visual Studio Code		Редактор кода, в котором удобно открывать папку проекта, создавать файлы и запускать встроенный терминал.
Браузер		Используется для отображения React-приложения и просмотра JSON-ответов сервера.
PowerShell		Командная оболочка Windows, через которую выполняются команды cd, mkdir, node, npm и другие.

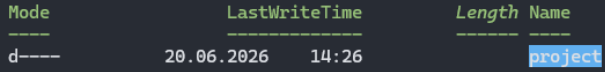
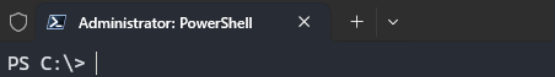
2.1. Работа с PowerShell и командой cd

Перед созданием проекта необходимо понимать, что команды в терминале выполняются не абстрактно, а в конкретной текущей папке. Текущий путь обычно отображается слева от символа >. Например, строка:

```
PS C:\Users\Student\Desktop\project>
```

означает, что терминал сейчас находится в папке project.

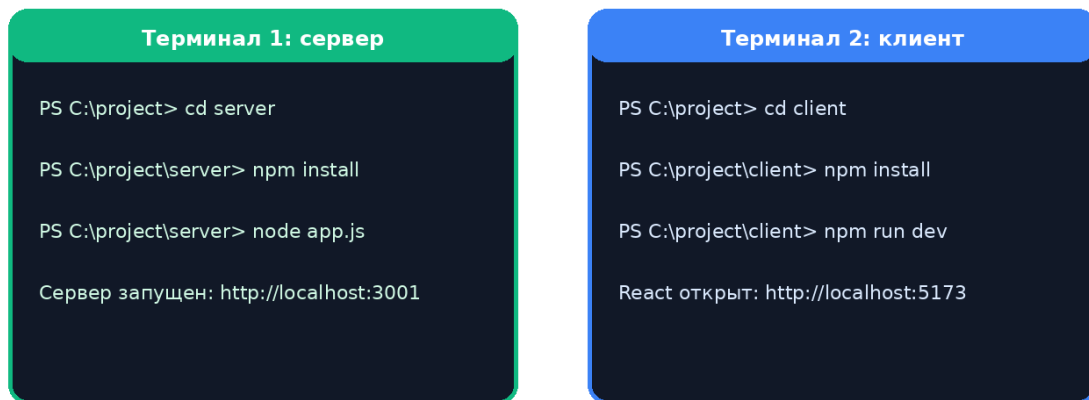
Команда	Что означает	Пример
pwd	Показывает текущую папку. Полезно, если студент не понимает, где сейчас находится терминал.	<pre>PS C:\> pwd Path ---- C:\</pre>
ls	Показывает содержимое текущей папки: файлы и подпапки.	<pre>PS C:\> ls Directory: C:\ Mode LastWriteTime Length Name ---- - d----- 07.04.2026 9:37 SWINDOWS.~BT d----- 20.01.2026 23:23 AppServ d----- 12.03.2026 21:49 CCD d----- 22.05.2026 19:01 files d----- 16.01.2026 21:53 inetpub d----- 01.04.2024 10:26 PerfLogs d-r----- 19.06.2026 0:12 Program Files d-r----- 16.05.2026 15:19 Program Files (x86) d----- 12.07.2025 16:26 Programs d----- 11.07.2025 0:06 QN d----- 16.05.2026 15:14 SCAD Soft d----- 20.12.2025 5:12 SwSetup d-r----- 26.03.2026 0:11 Users d----- 16.06.2026 23:42 windows d----- 20.01.2026 0:06 windows.old d----- 28.10.2025 22:05 XboxGames -a----- 23.03.2026 2:23 35195995 analogues.csv -a----- 27.01.2026 6:18 112496 appverifui.dll -a----- 27.01.2026 6:18 68120 vfcompat.dll</pre>

mkdir project	Создаёт новую папку project внутри текущей папки.	<pre>PS C:\> mkdir project</pre> 
cd server	Переходит в папку server, если она находится внутри текущей папки.	<pre>PS C:\> cd server PS C:\server> </pre>
cd ..	Поднимается на один уровень выше. Например, из project\server в project. Пример:	<pre>PS C:\files> cd .. PS C:\></pre>
cd D:\learning\SummerPractice2026\node	Переходит по полному пути к папке project.	<pre>PS C:\> cd D:\learning\SummerPractice2026\node PS D:\learning\SummerPractice2026\node> </pre>
cd "my project"	Переходит в папку, в названии которой есть пробел. Кавычки обязательны.	<pre>PS C:\> cd "my project" PS C:\my project> </pre>
clear	Очищает окно терминала от старых сообщений.	
Ctrl + C	Останавливает запущенный процесс: сервер или React-приложение. (выполняется сочетанием клавиш при активном окне PowerShell)	

Важно. Если команда `npm install` или `node app.js` выполняется не в той папке, проект не запустится. Перед запуском нужно убедиться, что терминал находится в нужной папке.

2.2. Два терминала для одного клиент-серверного приложения

Клиент-серверное приложение состоит из двух самостоятельных частей. Серверная часть запускается отдельно, а клиентская часть запускается отдельно. Поэтому во время проверки приложения обычно открывают два терминала: один для сервера, второй для React-клиента.



Если закрыть терминал сервера, React-клиент перестанет получать данные.

Рисунок 1 - Одновременный запуск сервера и клиента в двух терминалах

В первом терминале запускается сервер. Он должен продолжать работать, пока клиент отправляет запросы. Во втором терминале запускается React-клиент. Если закрыть серверный терминал или остановить сервер сочетанием Ctrl + C, клиентская часть останется открытой, но перестанет получать данные с сервера.

3. Клиент-серверная архитектура и технологический стек

3.1. Клиент-серверная архитектура

Клиент-серверная архитектура — это модель построения программного обеспечения, при которой приложение разделяется на две основные части: клиентскую и серверную. Клиентская часть отвечает за взаимодействие с пользователем, отображение интерфейса и отправку запросов. Серверная часть принимает эти запросы, выполняет обработку данных, применяет бизнес-логику и возвращает результат клиенту.

В веб-приложениях клиентской частью чаще всего является интерфейс, открытый в браузере. Пользователь нажимает кнопки, вводит данные в формы, переходит между страницами и видит результат работы приложения. Однако само хранение данных, проверка прав доступа, обработка форм и взаимодействие с базой данных выполняются на сервере.

Такое разделение делает систему более устойчивой и удобной для развития. Интерфейс можно изменять отдельно от серверной логики, а сервер можно расширять без полной переработки клиентской части. Кроме того, критически важные операции лучше выполнять на сервере, потому что код клиентской части доступен пользователю в браузере и не должен содержать секретные ключи, пароли или правила безопасности.

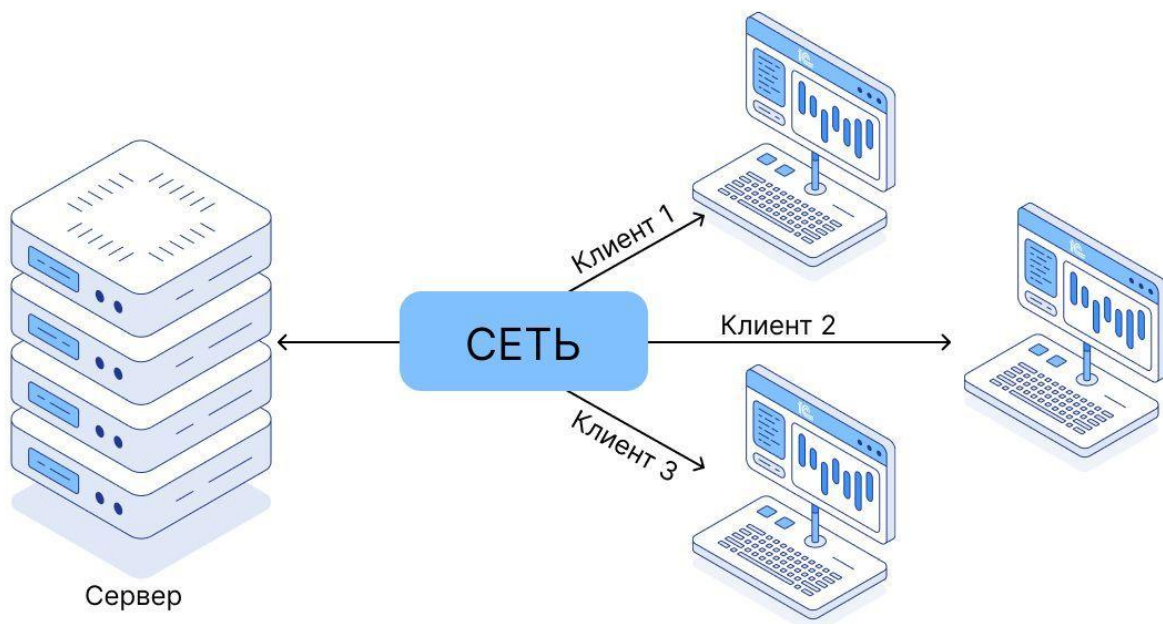


Рисунок 2 - Общая схема клиент-серверного взаимодействия

3.2. Роль клиентской части на React

React используется для создания клиентской части приложения. Его основная задача - построение пользовательского интерфейса из компонентов. Компонентами могут быть страницы, кнопки, формы, карточки, таблицы, модальные окна и другие элементы интерфейса.

Клиентская часть на React не должна выполнять всю работу приложения самостоятельно. Она отвечает за отображение данных, реакцию на действия пользователя и отправку запросов к серверу. Например, если пользователю необходимо получить список записей, React-компонент отправляет запрос к API, получает ответ и отображает данные на странице.

Важной особенностью React является автоматическое обновление интерфейса при изменении состояния компонента. Это позволяет получать данные с сервера, записывать их в состояние и сразу выводить обновлённую информацию без перезагрузки всей страницы.

3.3. Роль серверной части на Node.js

Node.js — это среда выполнения JavaScript вне браузера. Благодаря Node.js JavaScript можно использовать не только для клиентской части, но и для серверной разработки. Сервер на Node.js принимает HTTP-запросы, обрабатывает маршруты, работает с файлами, базами данных и внешними сервисами, а затем возвращает клиенту ответ.

В рамках данной методички Node.js используется в двух разных ролях. Во-первых, он нужен для запуска инструментов React-разработки: установки зависимостей, сборки проекта и запуска локального сервера разработки. Во-вторых, Node.js используется как серверная платформа, на которой будет выполняться код API.

Контекст использования	Что делает Node.js	Пример команды
Клиентская часть React	Запускает инструменты разработки, устанавливает зависимости и поднимает dev-сервер React-приложения.	npm install, npm run dev
Серверная часть	Выполняет JavaScript-код сервера, принимает HTTP-запросы и возвращает ответы клиенту.	node app.js, npm start

3.4. Технологический стек проекта

Технологический стек — это набор технологий, библиотек и инструментов, которые используются для разработки приложения. В данном методическом указании применяется связка React и Node.js.

- React используется для создания клиентского интерфейса веб-приложения;
- Node.js используется для запуска инструментов клиентской разработки и выполнения серверного JavaScript-кода;
- Express используется для удобного создания серверных маршрутов;
- JSON используется как основной формат обмена данными между клиентом и сервером;
- npm используется для установки зависимостей и запуска команд проекта.

Такой стек удобен для обучения, потому что клиентская и серверная части пишутся на одном языке - JavaScript. При этом важно понимать, что код React выполняется в браузере пользователя, а код Node.js выполняется на сервере.

3.5. Как клиент и сервер обмениваются данными

Связь между клиентом и сервером выполняется через HTTP-запросы и HTTP-ответы. Когда React-клиенту нужны данные, он отправляет запрос на определённый адрес сервера. Сервер принимает запрос, выбирает подходящий обработчик, формирует ответ и возвращает его клиенту.

HTTP-запрос обычно содержит метод, адрес, заголовки и при необходимости тело запроса. Метод показывает действие, которое клиент хочет выполнить. Например, GET чаще используется для получения данных, POST - для отправки новых данных, PUT или PATCH - для изменения данных, DELETE - для удаления.

HTTP-ответ содержит статус выполнения запроса, заголовки и тело ответа. Если запрос выполнен успешно, сервер часто возвращает статус 200 OK. Если адрес не найден, сервер может вернуть 404 Not Found. Если на сервере произошла ошибка, может быть возвращён статус 500 Internal Server Error.

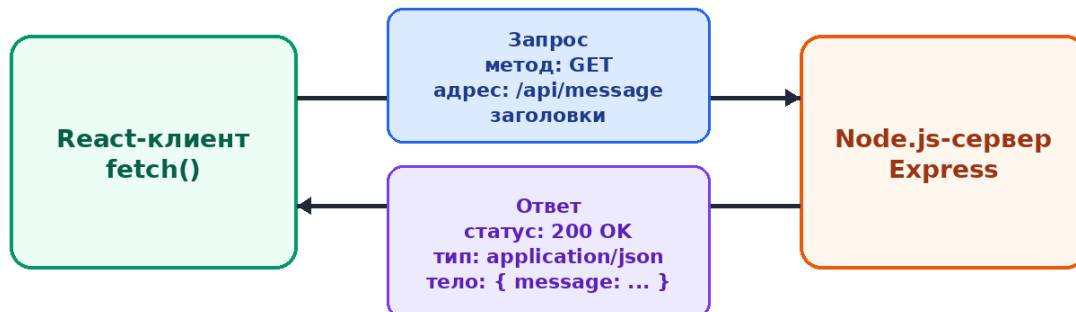


Рисунок 3 - Упрощённая схема HTTP-запроса и HTTP-ответа

Термин	Пояснение	Пример
URL	Адрес, по которому клиент обращается к серверу.	http://localhost:3001/api/message
Порт	Числовой идентификатор приложения на компьютере.	3001
Маршрут	Часть адреса, для которой на сервере создаётся обработчик.	/api/message
API	Набор маршрутов сервера, через которые клиент получает или отправляет данные.	GET /api/message
JSON	Текстовый формат передачи структурированных данных.	{ "message": "text" }

3.6. Зачем нужен Express

Встроенный модуль **http** показывает, как работает сервер на самом базовом уровне. С его помощью можно создать приложение, которое принимает запрос и отправляет ответ. Однако при росте проекта ручная обработка адресов, методов и заголовков становится неудобной.

Express упрощает эту работу. Он позволяет описывать маршруты в виде **app.get()**, **app.post()** и других методов. Благодаря этому код становится более понятным: сразу видно, какой обработчик отвечает за конкретный адрес и конкретный HTTP-метод.

В данной методичке сначала рассматривается простой сервер на **http**, а затем создаётся практический сервер на Express. Такой порядок позволяет понять базовый механизм **request/response** и затем перейти к более удобному инструменту.

4. Установка и первичная проверка Node.js

Для установки проще всего загрузить с официального сайта установочный пакет и запустить его. Для этого перейдем на официальный сайт <https://nodejs.org/en/>. На главной странице мы сразу увидим две возможные опции для загрузки: самая последняя версия NodeJS и LTS-версия.

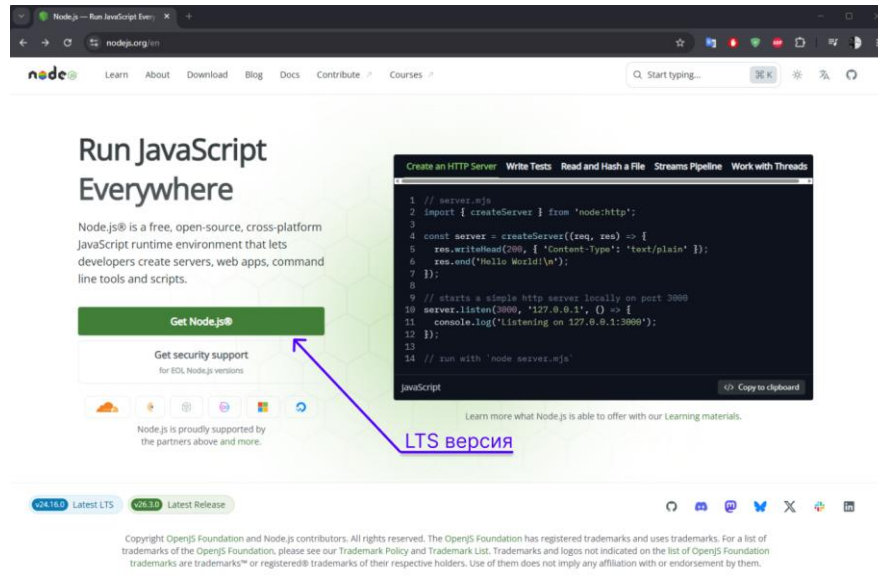


Рисунок 4 – Страница для скачивания Node.js

Причем в зависимости от текущей операционной системы при нажатии на кнопку/ссылку будет загружаться соответствующий установочный пакет: для Windows это файл .msi.

Загрузим LTS-версию. В моем случае это версия 22.17.0. На Windows после запуска откроется следующая программа установщика:

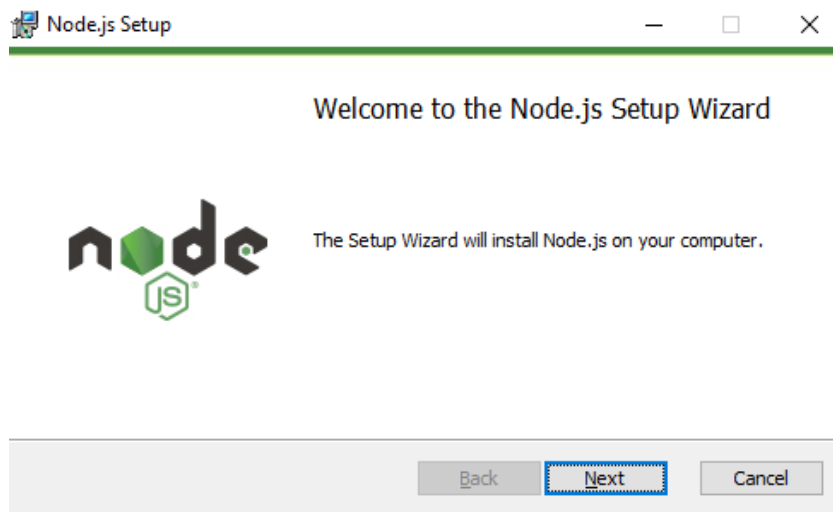


Рисунок 5 – Процесс установки Node.js

4.1. Проверка установки

После успешной установки вы можем ввести в командной строке/терминале команду `node -v`, и нам отобразится текущая версия `node.js`:



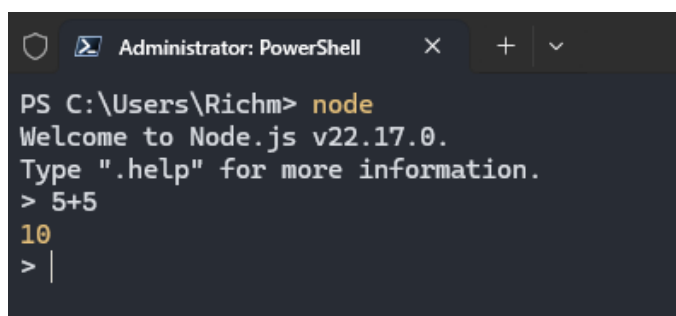
```
Administrator: PowerShell
PowerShell 7.6.2
PS C:\Users\Richm> node -v
v22.17.0
PS C:\Users\Richm> |
```

Рисунок 6 – Проверка установленной версии Node.js

4.2. Запуск и использование

После установки NodeJS нам становится доступным такой инструмент как REPL. REPL (Read Eval Print Loop) представляет возможность запуска выражений на языке JavaScript в командной строке или терминале.

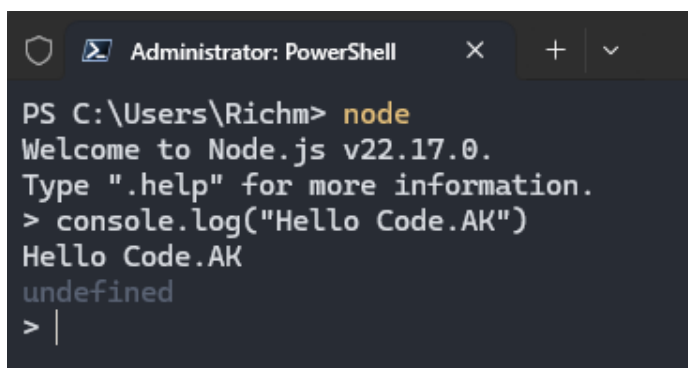
Так, запустим power shell (на Windows) и введем команду `node`. После ввода этой команды мы можем выполнять различные выражения на JavaScript:



```
Administrator: PowerShell
PS C:\Users\Richm> node
Welcome to Node.js v22.17.0.
Type ".help" for more information.
> 5+5
10
> |
```

Рисунок 7 – Выполнение расчета на JavaScript при помощи Node.js

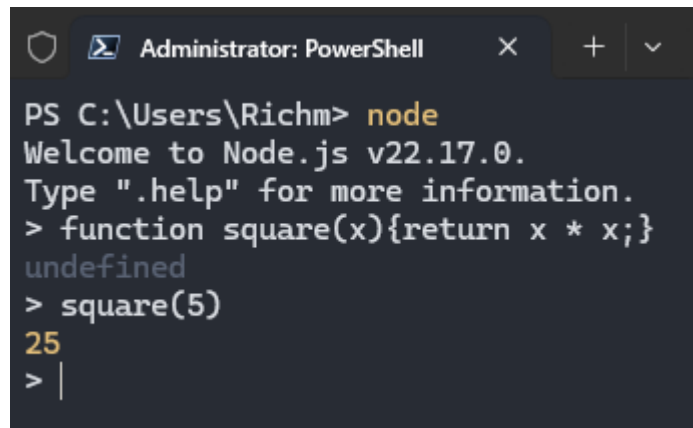
Или используем какую-нибудь функцию JS:



```
Administrator: PowerShell
PS C:\Users\Richm> node
Welcome to Node.js v22.17.0.
Type ".help" for more information.
> console.log("Hello Code.AK")
Hello Code.AK
undefined
> |
```

Рисунок 8 – Выполнение функции на JavaScript при помощи Node.js

Можно определять свои функции и затем их вызывать, например, возведение числа в квадрат:

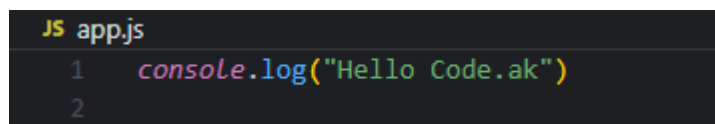


```
Administrator: PowerShell
PS C:\Users\Richm> node
Welcome to Node.js v22.17.0.
Type ".help" for more information.
> function square(x){return x * x;}
undefined
> square(5)
25
> |
```

Рисунок 9 – Выполнение собственной функции на JavaScript при помощи Node.js

4.3. Выполнение файла

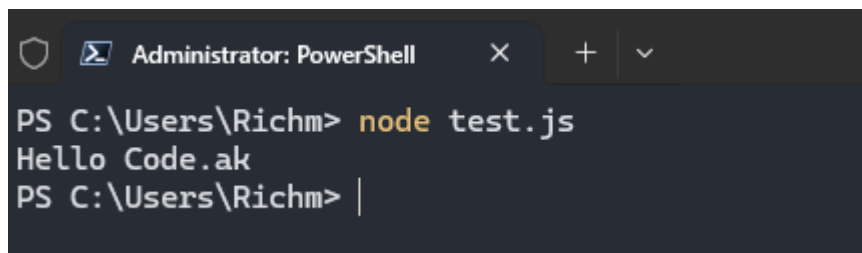
Вместо того чтобы вводить весь код напрямую в консоль, удобнее вынести его во внешний файл. Например, создадим на жестком диске новый каталог, допустим, **C:\node**, в который поместим новый файл **test.js** со следующим кодом:



```
JS app.js
1 console.log("Hello Code.ak")
2
```

Рисунок 10 – Создание внешнего файла для Node.js

В power shell перейдем с помощью команды **cd** к каталогу, в котором расположен файл, а затем выполним команду **node test.js**. Данная команда выполнит код из файла **test.js**:



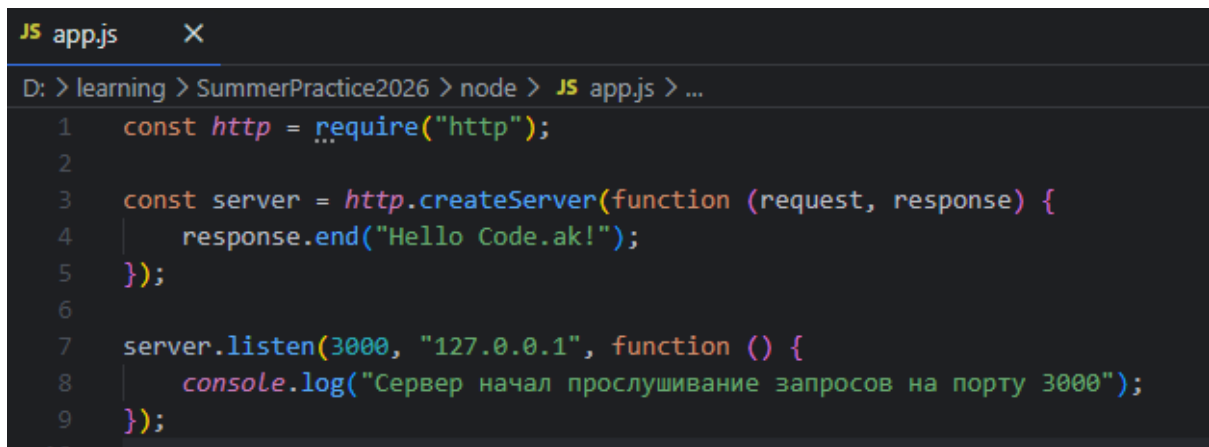
```
Administrator: PowerShell
PS C:\Users\Richm> node test.js
Hello Code.ak
PS C:\Users\Richm> |
```

Рисунок 11 – Выполнение внешнего файла для Node.js

5. Создание простого HTTP-сервера

Теперь создадим первое серверное приложение без дополнительных библиотек. Для этого используется встроенный модуль **http**. Он входит в состав **Node.js** и позволяет создать сервер, который принимает запросы от браузера.

Создайте папку для проекта и файл **app.js**. В файл **app.js** добавьте следующий код:



```
JS app.js X
D: > learning > SummerPractice2026 > node > JS app.js > ...
1  const http = require("http");
2
3  const server = http.createServer(function (request, response) {
4    response.end("Hello Code.ak!");
5  });
6
7  server.listen(3000, "127.0.0.1", function () {
8    console.log("Сервер начал прослушивание запросов на порту 3000");
9  });
```

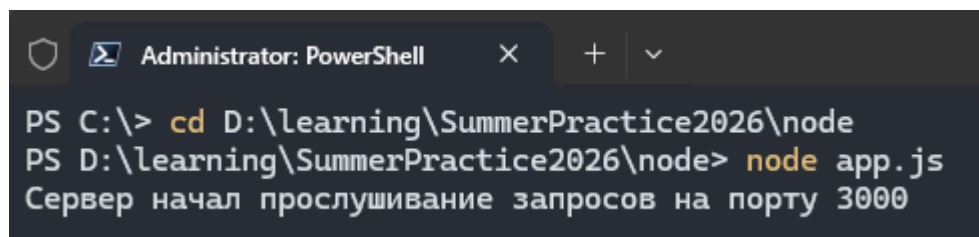
Рисунок 12 – Написание простого сервера

В первой строке подключается модуль **http**. Затем с помощью метода **createServer()** создаётся сервер. Внутри функции-обработчика доступны два параметра: **request** и **response**. Параметр **request** содержит информацию о входящем запросе, а **response** используется для отправки ответа.

Метод **response.end()** завершает формирование ответа и отправляет его клиенту. Метод **listen()** запускает сервер на указанном порту. В данном примере сервер будет доступен по адресу **http://localhost:3000/**.

5.1. Запуск сервера

Для запуска сервера необходимо перейти в папку с файлом **app.js** и выполнить команду:



```
Administrator: PowerShell X + v
PS C:\> cd D:\learning\SummerPractice2026\node
PS D:\learning\SummerPractice2026\node> node app.js
Сервер начал прослушивание запросов на порту 3000
```

Рисунок 13 – Запуск простого сервера на Node.js

Если сервер запущен успешно, в терминале появится сообщение о прослушивании порта. После этого необходимо открыть браузер и перейти по адресу:

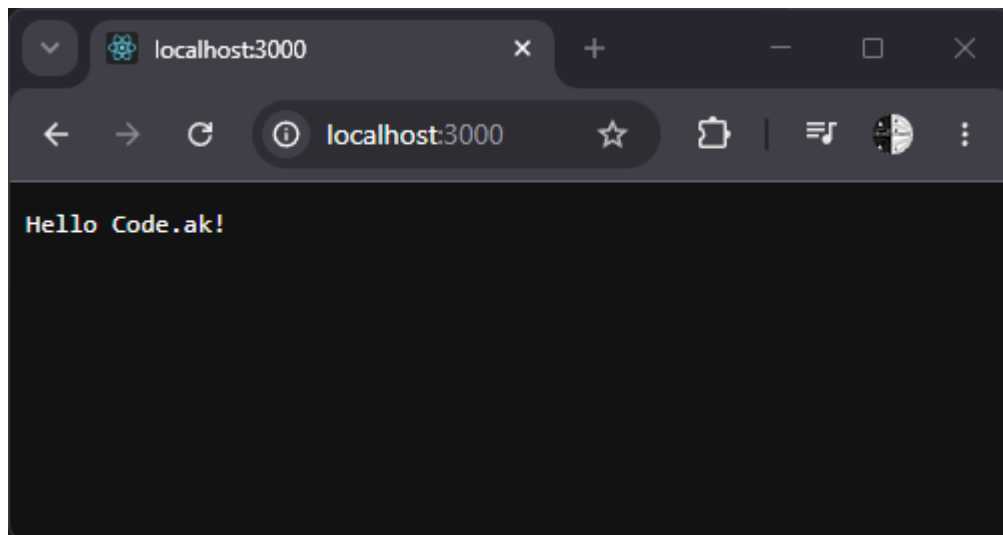


Рисунок 14 – Проверка работоспособности простого сервера на Node.js

На странице должно отобразиться сообщение «Hello Code.ak!».

5.2. Объект request

Параметр request содержит информацию о запросе, который пришёл от клиента. С его помощью можно узнать адрес запроса, HTTP-метод и заголовки. Это полезно, когда сервер должен по-разному реагировать на разные запросы. Добавим вывод в консоль объекта request.

```
JS app.js X
D: > learning > SummerPractice2026 > node > JS app.js > ...
1  const http = require("http");
2
3  http.createServer(function (request, response) {
4    console.log("URL:", request.url);
5    console.log("Метод запроса:", request.method);
6    console.log("User-Agent:", request.headers["user-agent"]);
7
8    response.end("Информация о запросе выведена в консоль");
9  }).listen(3000, function () {
10   console.log("Сервер запущен: http://localhost:3000");
11 });
```

Рисунок 15 – Добавление вывода объекта request в логику работы сервера на Node.js

Свойство request	Назначение
request.url	Показывает адрес, к которому обратился клиент.
request.method	Показывает HTTP-метод запроса: GET, POST, PUT, DELETE и т.д.
request.headers	Содержит заголовки запроса, например информацию о браузере.

В результате добавления вывода в консоль и перезагрузки страницы в браузере, в консоли отобразится следующее:

```

Administrator: PowerShell
PS D:\learning\SummerPractice2026\node> node app.js
Сервер запущен: http://localhost:3000
URL: /
Метод запроса: GET
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36
URL: /favicon.ico
Метод запроса: GET
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36

```

Рисунок 16 – Запуск сервера на Node.js и проверка отображения объекта request

5.3. Объект response

Параметр response управляет отправкой ответа клиенту. Через него можно задать статус ответа, заголовки, тип содержимого и тело ответа. Добавим вывод в консоль объекта response.

```

JS app.js
D: > learning > SummerPractice2026 > node > JS app.js > ...
1  const http = require("http");
2
3  http.createServer(function (request, response) {
4      response.statusCode = 200;
5      response.setHeader("Content-Type", "text/plain; charset=utf-8");
6      response.write("Text 1\n");
7      response.write("Text 2\n");
8      response.end("End");
9  }).listen(3000);
10

```

Рисунок 17 – Добавление вывода объекта response в логику работы сервера на Node.js

Метод или свойство <code>response</code>	Назначение
<code>statusCode</code>	Устанавливает HTTP-статус ответа.
<code>setHeader(name, value)</code>	Добавляет заголовок ответа.
<code>write()</code>	Записывает часть тела ответа.
<code>end()</code>	Завершает ответ и отправляет его клиенту.

Простой сервер на модуле `http` полезен для понимания, но в реальных проектах удобнее использовать `Express`. Далее будет создан сервер, который сможет работать как API для React-клиента.

5.4. Создание рабочего сервера для React-клиента

5.4.1. Переход от `http` к `Express`

Express — это фреймворк для **Node.js**, который упрощает создание серверных приложений и API. С его помощью можно описывать маршруты более коротко и понятно, чем при использовании встроенного модуля `http`.

Для React-клиента сервер обычно нужен не для вывода HTML-страниц, а для выдачи данных. Поэтому серверная часть будет возвращать JSON-ответы по API-маршрутам.

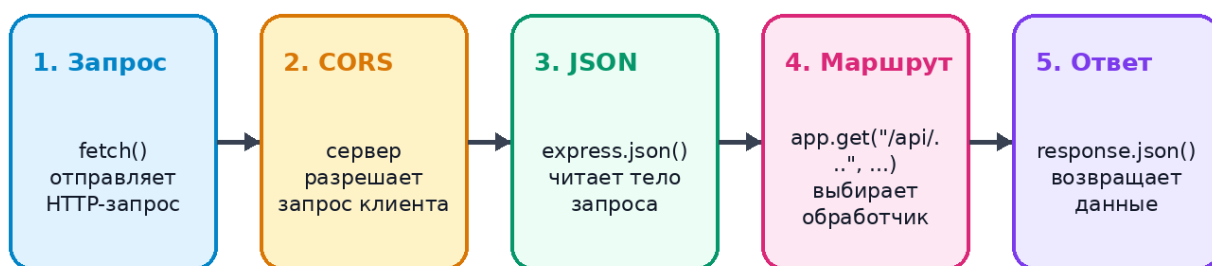


Рисунок 18 - Обработка запроса во фреймворке `Express`

5.4.2. Структура проекта

Создадим проект, в котором клиентская и серверная части находятся в отдельных папках. Это позволит запускать их независимо и не смешивать код интерфейса с серверной логикой.



Рисунок 19 – Рекомендуемая структура проекта

5.4.3. Создание серверной части

В корневой папке проекта создайте папку server и перейдите в неё:

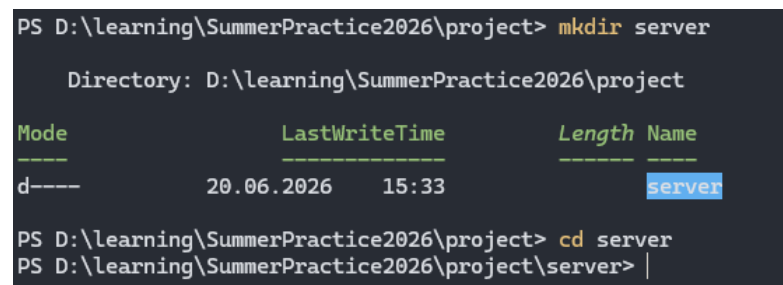


Рисунок 20 – Создание и переход в папку server

Инициализируйте Node.js-проект:

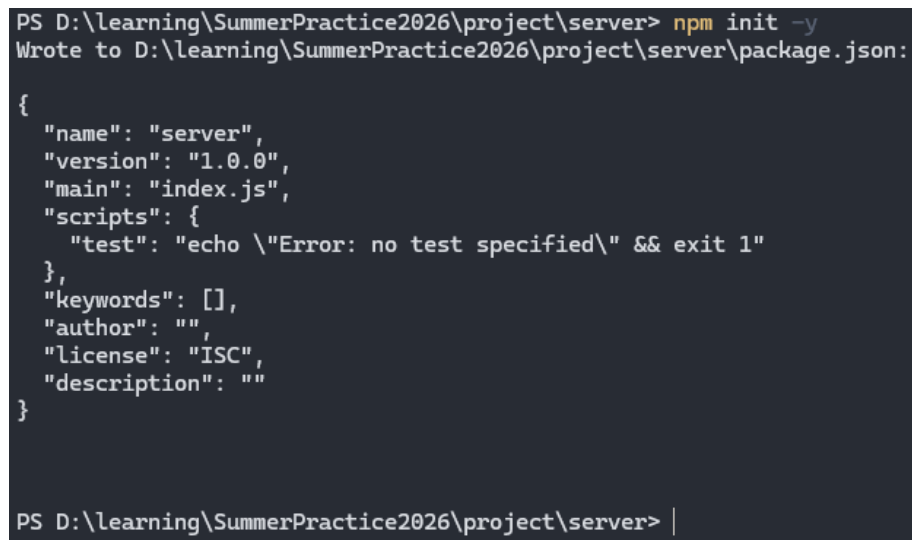


Рисунок 21 – Инициализация проекта

После выполнения команды появится файл **package.json**. Далее установите зависимости:

```
PS D:\learning\SummerPractice2026\project\server> npm install express cors
added 69 packages, and audited 70 packages in 3s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\learning\SummerPractice2026\project\server> |
```

Рисунок 22 – Установка зависимостей

- **express** нужен для создания сервера и маршрутов;
- **cors** нужен для разрешения запросов между React-клиентом и Node.js-сервером, если они запущены на разных портах.

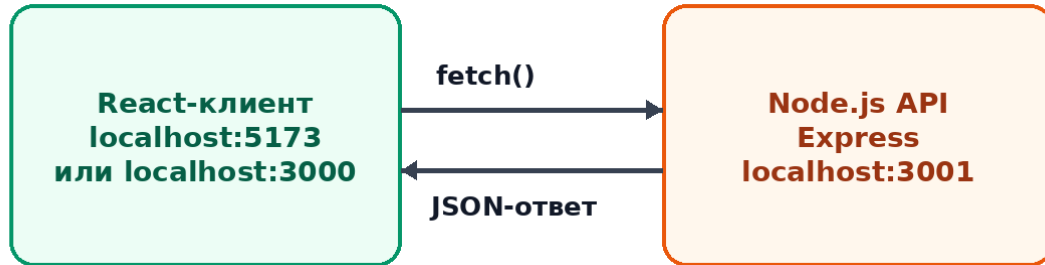
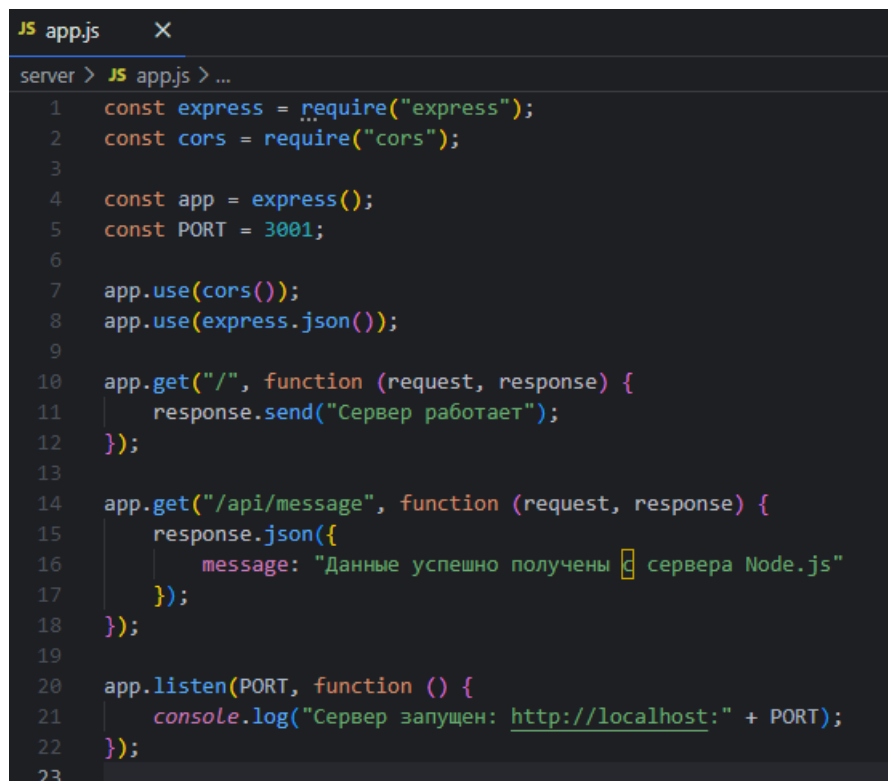


Рисунок 23 - Разные порты клиента и сервера при локальной разработке

Важно. Порт — это условный числовой идентификатор, который позволяет операционной системе понять, какой именно программе необходимо передать входящий запрос. Вместе с адресом компьютера порт образует точку подключения к приложению.

5.4.4. Создание файла **app.js**

В папке **server** создайте файл **app.js**. В нём будет находиться основной код серверной части.



```

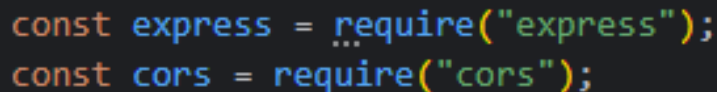
JS app.js X
server > JS app.js > ...
1  const express = require("express");
2  const cors = require("cors");
3
4  const app = express();
5  const PORT = 3001;
6
7  app.use(cors());
8  app.use(express.json());
9
10 app.get("/", function (request, response) {
11   response.send("Сервер работает");
12 });
13
14 app.get("/api/message", function (request, response) {
15   response.json({
16     message: "Данные успешно получены с сервера Node.js"
17   });
18 });
19
20 app.listen(PORT, function () {
21   console.log("Сервер запущен: http://localhost:" + PORT);
22 });
23

```

Рисунок 24 – Пример написания файла app.js

5.4.5.Разбор кода сервера

В первых строках подключаются модули **express** и **cors**. Модуль **express** создаёт серверное приложение, а **cors** разрешает запросы от клиентского приложения.



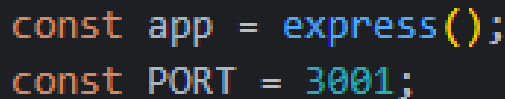
```

const express = require("express");
const cors = require("cors");

```

Рисунок 25 – Модули express и cors

Затем создаётся объект **app** и задаётся порт сервера.



```

const app = express();
const PORT = 3001;

```

Рисунок 26 – Задание порта и объекта app

Порт **3001** выбран для того, чтобы сервер не конфликтовал с клиентской частью. React-приложение может использовать порт **3000** или **5173**, поэтому сервер лучше запускать отдельно.

Далее подключаются промежуточные обработчики **middleware**:

```
app.use(cors());  
app.use(express.json());
```

Рисунок 27 – Подключение обработчиков

- `cors()` разрешает запросы с другого порта;
- `express.json()` позволяет серверу читать JSON-данные из тела запроса.

Первый маршрут используется для проверки работы сервера:

```
app.get("/", function (request, response) {  
  response.send("Сервер работает");  
});
```

Рисунок 28 – Написание маршрута “/”

Второй маршрут является API-маршрутом для React-клиента. Он возвращает JSON-объект:

```
app.get("/api/message", function (request, response) {  
  response.json({  
    message: "Данные успешно получены с сервера Node.js"  
  });  
});
```

Рисунок 29 – Написание маршрута “/api/message”

Метод **`response.json()`** отправляет клиенту данные в формате JSON. React сможет получить эти данные через `fetch` и вывести значение `message` на экран.

5.4.6. Настройка команды запуска сервера

Сервер можно запускать командой **`node app.js`**. Однако удобнее добавить команду запуска в **`package.json`**. В разделе `scripts` необходимо указать:

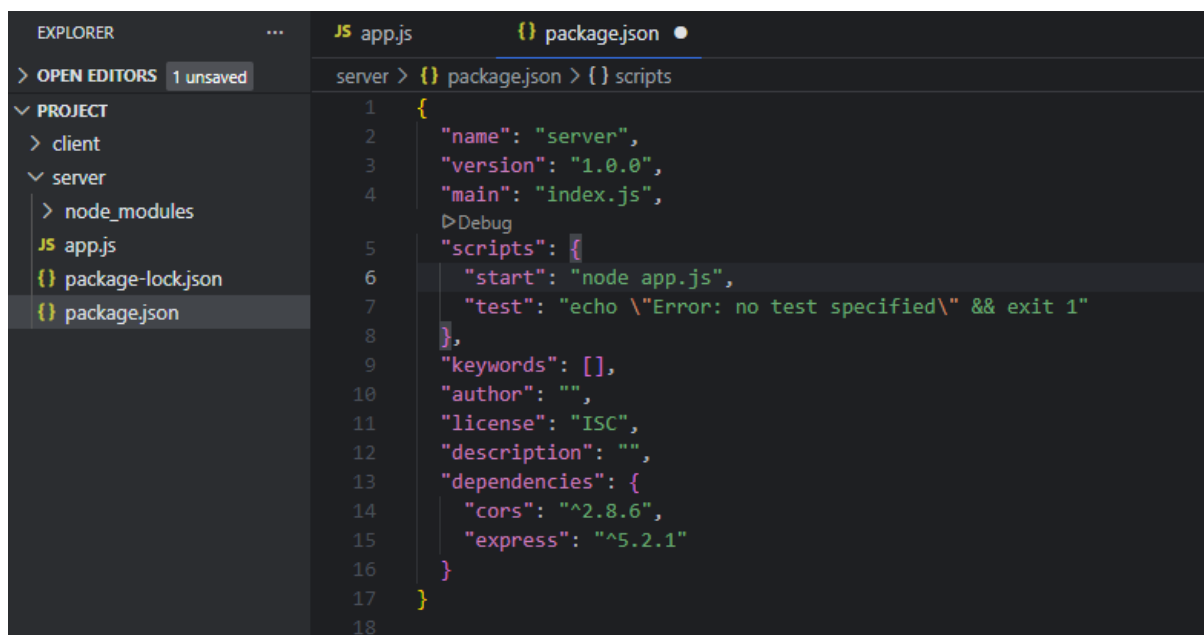


Рисунок 30 – Добавление команды для запуска в package.json

Теперь сервер можно запускать командой «npm start».

5.4.7. Проверка работы сервера

Для проверки перейдите в папку server и выполните команду запуска:

```
PS D:\learning\SummerPractice2026\project> cd server
PS D:\learning\SummerPractice2026\project\server> npm start

> server@1.0.0 start
> node app.js

Сервер запущен: http://localhost:3001
```

Рисунок 31 – Запуск подготовленного сервера

После успешного запуска откройте в браузере адрес **http://localhost:3001/**:

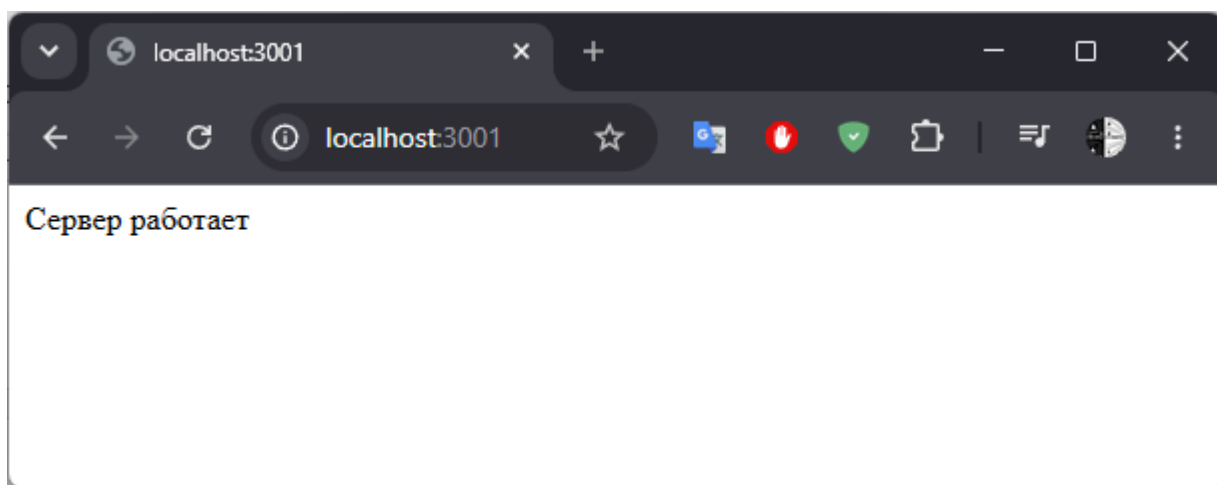


Рисунок 32 – Проверка работы сервера в браузере

На странице должно появиться сообщение «**Сервер работает**».

Затем проверьте API-маршрут:

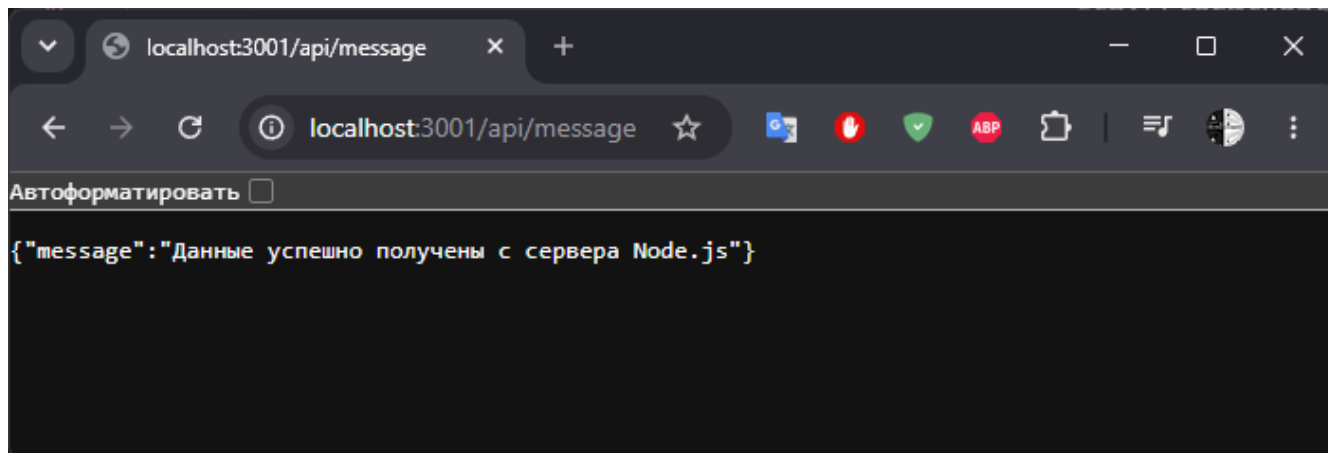


Рисунок 33 – Проверка работы API маршрута сервера в браузере

Если JSON-ответ отображается в браузере, серверная часть работает корректно и готова к подключению React-клиента.

6. Создание клиентской части на React

После создания серверной части необходимо подготовить клиентскую часть приложения. Клиентская часть будет отвечать за отображение интерфейса, отправку HTTP-запросов на сервер и вывод полученных данных пользователю. В данном разделе создаётся React-приложение, которое будет работать совместно с ранее созданным сервером Node.js + Express.

В качестве инструмента создания проекта используется Vite. Он создаёт минимальный шаблон React-приложения, поднимает локальный сервер разработки и позволяет быстро проверять изменения в браузере. Для начинающего разработчика такой вариант удобен тем, что не требует ручной настройки сборщика проекта.



Рисунок 34 – Этапы создания клиентской части на React

6.1. Назначение клиентской части

Клиентская часть является той частью приложения, с которой непосредственно взаимодействует пользователь. Она отображает страницы, кнопки, формы, сообщения об ошибках и результаты выполнения запросов. В рамках учебного проекта клиент будет выполнять три основные задачи:

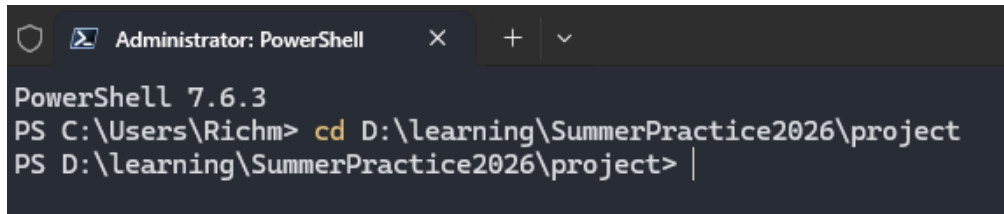
- запускаться в браузере как отдельное React-приложение;
- отправлять запрос на серверный API по адресу **http://localhost:3001**;
- получать JSON-ответ от сервера и отображать его на странице.

Важно понимать, что React-клиент сам по себе не является полноценным сервером. Он может отображать интерфейс и выполнять JavaScript-код в браузере, но хранение данных, проверка запросов и основная серверная логика должны находиться на стороне Node.js-сервера.

6.2. Создание React-проекта

Создавать клиентскую часть рекомендуется в общей папке проекта, рядом с папкой server. Если ранее была создана структура проекта с папкой server, то теперь в той же корневой папке необходимо создать папку client.

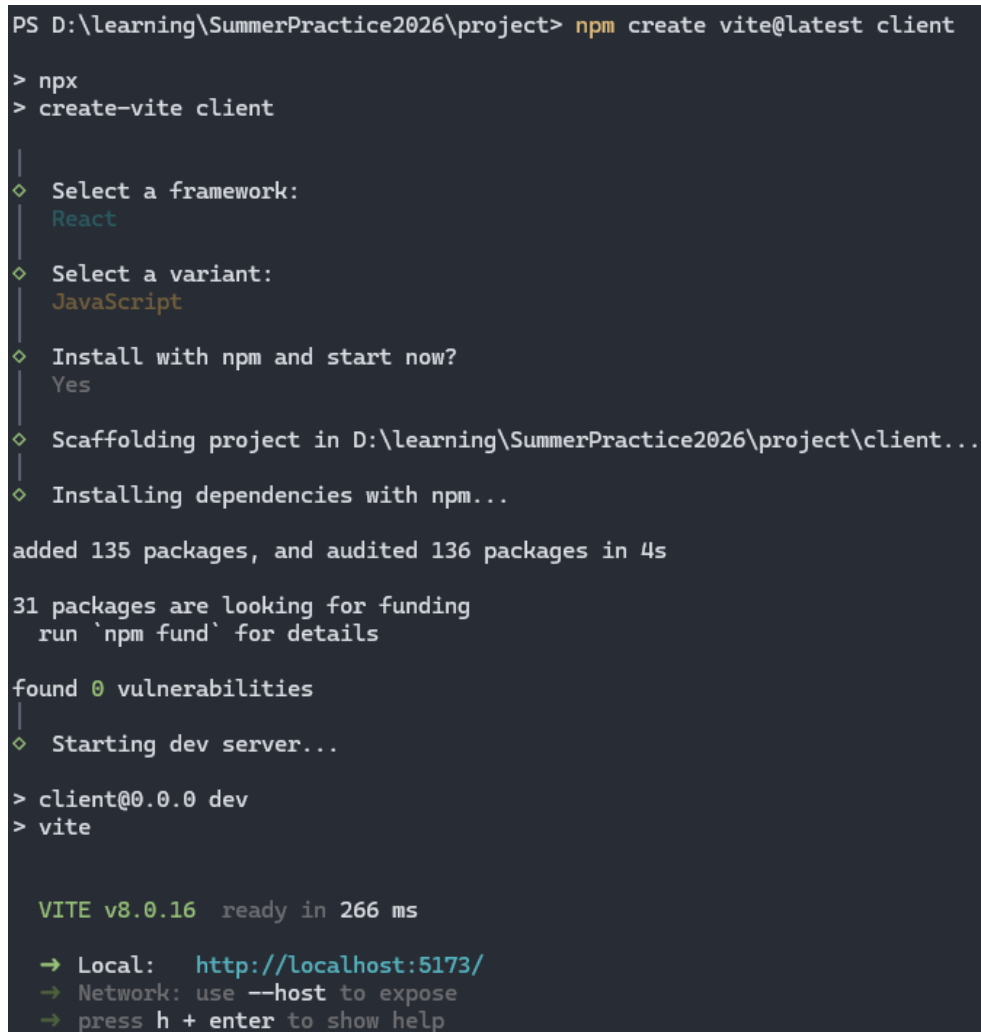
Перейдите в корневую папку проекта:



```
Administrator: PowerShell
PowerShell 7.6.3
PS C:\Users\Richm> cd D:\learning\SummerPractice2026\project
PS D:\learning\SummerPractice2026\project> |
```

Рисунок 35 – Переход в корневую папку проекта

После этого выполните команду создания React-приложения, на фразе выбора фреймворка выберите “React” и далее “JavaScript” как показано на фото:



```
PS D:\learning\SummerPractice2026\project> npm create vite@latest client
> npx
> create-vite client

Select a framework:
  React

Select a variant:
  JavaScript

Install with npm and start now?
  Yes

Scaffolding project in D:\learning\SummerPractice2026\project\client...

Installing dependencies with npm...

added 135 packages, and audited 136 packages in 4s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

Starting dev server...

> client@0.0.0 dev
> vite

VITE v8.0.16 ready in 266 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Рисунок 36 – Создание React-приложения

В данной команде слово `client` обозначает название папки, в которую будет установлен шаблон приложения. После выполнения команды, клиентская часть приложения будет автоматически запущена и доступна в браузере.

Если выбрать “No” на этапе “Install with npm and start now?”, то необходимо перейти в созданную папку и установить зависимости проекта:

```
PS D:\learning\SummerPractice2026\project> cd client
PS D:\learning\SummerPractice2026\project\client> npm install

up to date, audited 136 packages in 2s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\learning\SummerPractice2026\project\client> |
```

Рисунок 37 – Установка зависимостей клиентской части

Команда `npm install` скачивает все пакеты, которые перечислены в файле `package.json`. После её выполнения в проекте появляется папка `node_modules`, где находятся установленные зависимости.

Важно. Папку “`node_modules`” не нужно создавать вручную и не нужно переносить между компьютерами. Она восстанавливается автоматически командой `npm install`.

6.3. Запуск React-приложения

Для запуска клиентской части в режиме разработки используется команда:

```
PS D:\learning\SummerPractice2026\project\client> npm run dev

> client@0.0.0 dev
> vite

VITE v8.0.16 ready in 222 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Рисунок 38 – Запуск клиентской части

После запуска в терминале появится локальный адрес приложения. Чаще всего Vite запускает React-клиент по адресу:

`http://localhost:5173/`

Откройте этот адрес в браузере. Если проект создан корректно, на странице появится стандартный стартовый экран. Это означает, что клиентская часть успешно создана и может быть изменена под задачи учебного проекта.

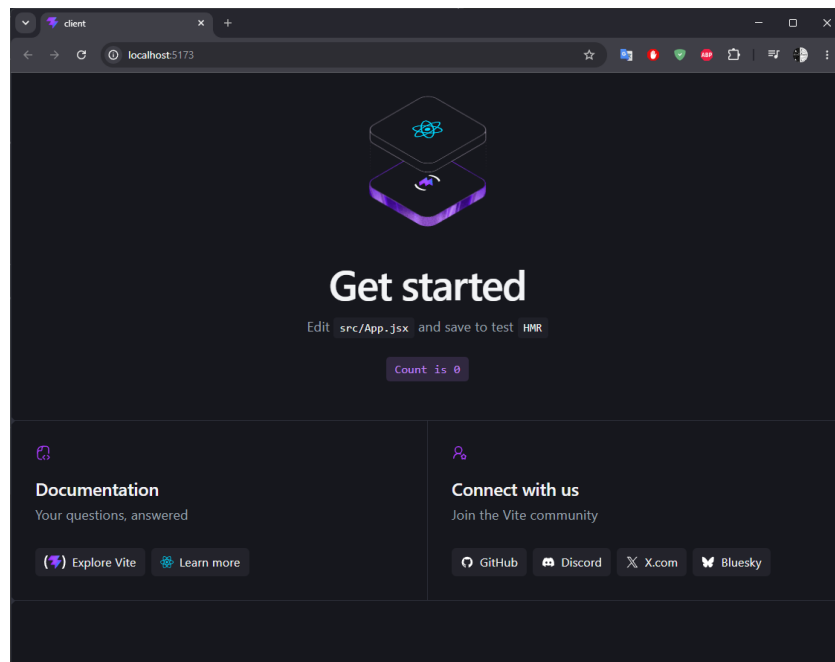


Рисунок 39 – Проверка в браузере клиентской части

Важно. Терминал с командой `npm run dev` должен оставаться открытым. Пока команда выполняется, локальный сервер разработки React-приложения продолжает работать.

6.4. Структура клиентского проекта

После создания React-приложения в папке `client` появится набор файлов и папок. На первом этапе не требуется подробно изучать каждую служебную часть проекта, но необходимо понимать назначение основных элементов.

client/	
├ index.html	входная HTML-страница
├ package.json	зависимости и команды запуска
├ node_modules/	установленные пакеты
├ public/	статические файлы
└ src/	исходный код приложения
├ main.jsx	точка входа React-приложения
├ App.jsx	главный компонент интерфейса
└ App.css	стили главного компонента

Рисунок 40 - Структура React-проекта

Элемент проекта	Назначение
index.html	Главная HTML-страница. В ней находится элемент с id="root", внутрь которого React помещает интерфейс приложения.
package.json	Файл с информацией о проекте, зависимостях и командах запуска.
src/main.jsx	Точка входа React-приложения. Здесь главный компонент App подключается к HTML-странице.
src/App.jsx	Главный компонент приложения. На начальном этапе именно в этом файле удобно писать основной интерфейс.
src/App.css	Файл стилей для компонента App.
public	Папка для статических файлов, которые должны быть доступны напрямую.

Основная работа в учебном примере будет выполняться в файлах **App.jsx** и **App.css**. Файл **main.jsx** обычно меняется редко, так как его задача заключается в подключении React-приложения к странице.

6.5. Точка входа React-приложения

Файл **src/main.jsx** является точкой входа клиентского приложения. Он получает HTML-элемент с идентификатором **root** и отрисовывает внутри него главный компонент **App**.

Пример содержимого файла **src/main.jsx**:

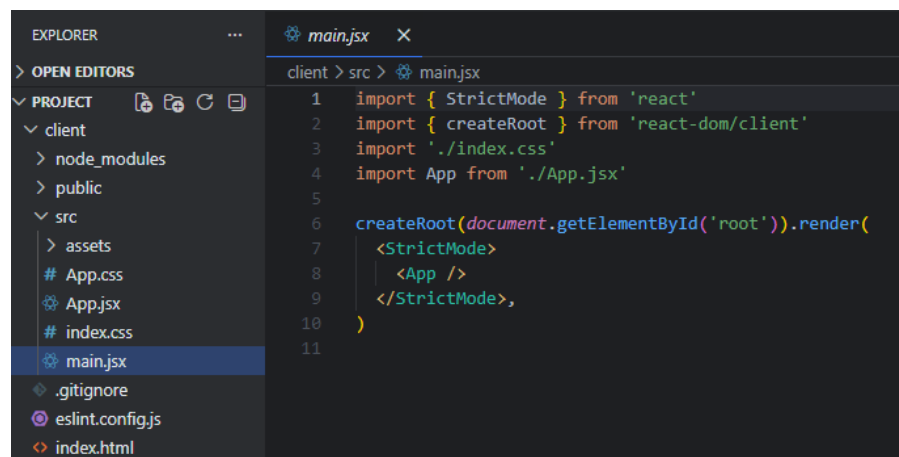


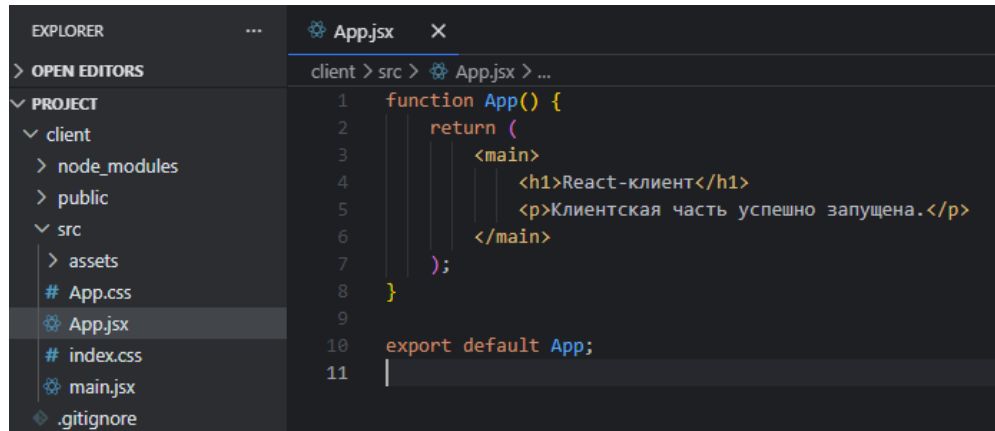
Рисунок 41 – Пример написания main.jsx

В данном коде компонент **App** подключается из файла **App.jsx**. Именно этот компонент будет содержать основной интерфейс учебного приложения.

6.6. Очистка стартового шаблона

Стартовый шаблон **Vite** содержит демонстрационный интерфейс с логотипами и счётчиком. Его лучше заменить на более понятный, связанный с серверной частью. Откройте файл **src/App.jsx** и удалите его исходное содержимое.

Затем создайте простой компонент:



```
1 function App() {
2   return (
3     <main>
4       <h1>React-клиент</h1>
5       <p>Клиентская часть успешно запущена.</p>
6     </main>
7   );
8 }
9
10 export default App;
11
```

Рисунок 42 – Пример написания компонента App

После сохранения файла страница в браузере автоматически обновится. Если на странице появился заголовок **React-клиент**, значит редактирование компонента **App.jsx** выполняется корректно.

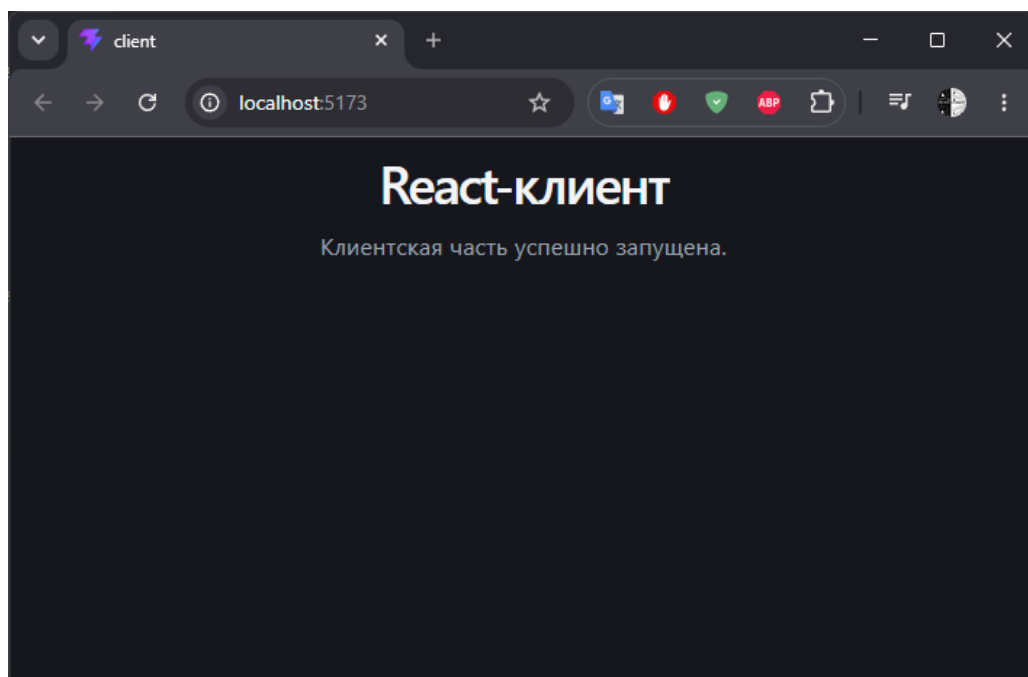


Рисунок 43 – Проверка работы написанного компонента в браузере

6.7. Настройка адреса сервера

Чтобы React-клиент мог обращаться к серверу, ему необходимо знать адрес API. В данном случае серверная часть запускается на порту 3001. Поэтому базовый адрес сервера имеет вид:

http://localhost:3001

Можно прописывать этот адрес прямо внутри fetch-запроса, однако более аккуратный вариант - вынести его в файл окружения. Для этого в корне папки client создайте файл .env и добавьте в него строку:

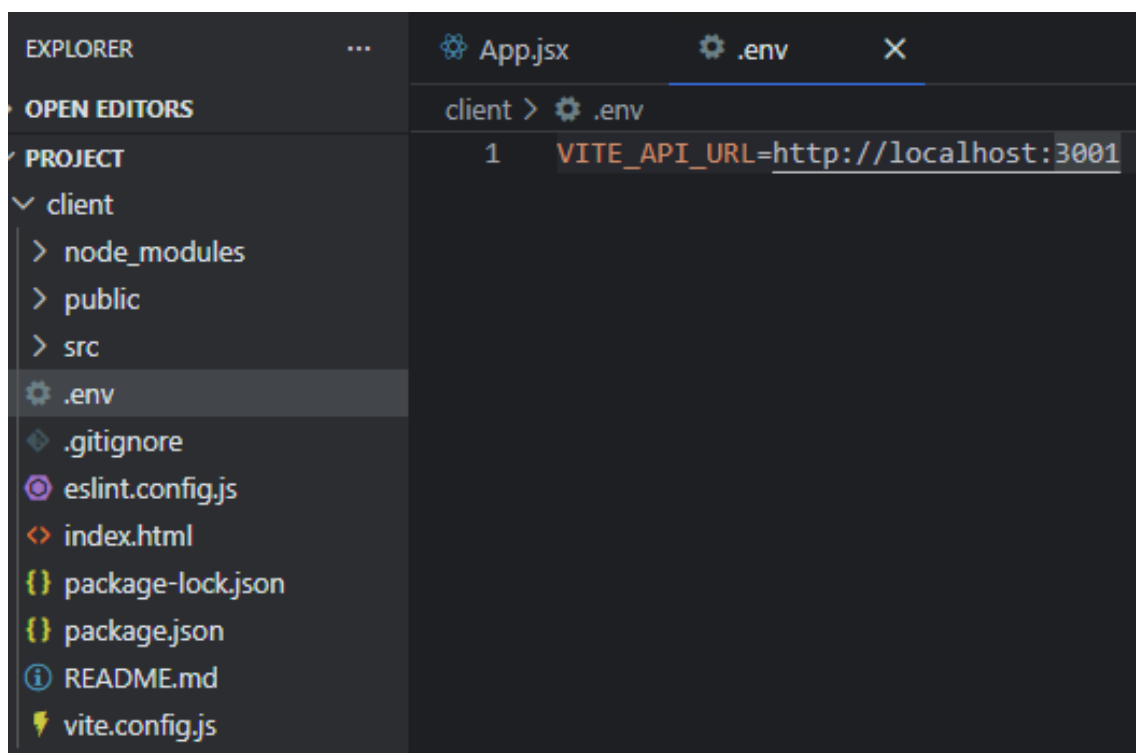


Рисунок 44 – Подготовка файла окружения

Переменные окружения в Vite, которые должны быть доступны в клиентском коде, должны начинаться с префикса **VITE_**. В приложении такая переменная будет доступна через **import.meta.env.VITE_API_URL**.

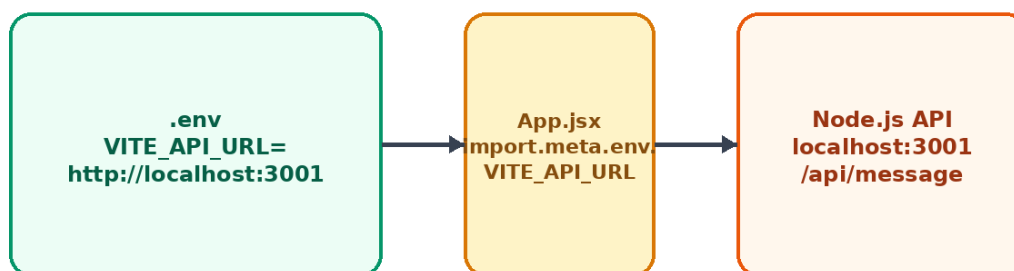


Рисунок 45 – Использование адреса сервера в React-приложении

Важно. После создания или изменения файла **.env** необходимо остановить React-приложение в терминале и запустить его снова командой **npm run dev**. Иначе новая переменная окружения может не примениться.

6.8. Получение данных с сервера через **fetch**

Теперь клиентскую часть можно подключить к серверу. Для этого в компоненте **App** используется функция **fetch()**, которая отправляет HTTP-запрос по адресу **/api/message**. Этот маршрут должен быть заранее реализован на сервере Node.js + Express.

Откройте файл **src/App.jsx** и перепишите его содержимое следующим образом:

```

1  import { useEffect, useState } from "react";
2  import "../App.css";
3
4  const API_URL = import.meta.env.VITE_API_URL || "http://localhost:3001";
5
6  function App() {
7    const [message, setMessage] = useState("");
8    const [status, setStatus] = useState("Ожидание ответа сервера");
9    const [loading, setLoading] = useState(false);
10   const [error, setError] = useState("");
11
12   async function loadMessage() {
13     try {
14       setLoading(true);
15       setError("");
16       setStatus("Выполняется запрос к серверу...");
17
18       const response = await fetch(API_URL + "/api/message");
19
20       if (!response.ok) {
21         throw new Error("Ошибка HTTP: " + response.status);
22       }
23
24       const data = await response.json();
25       setMessage(data.message);
26       setStatus("Ответ сервера получен");
27     } catch (err) {
28       setError("Не удалось получить данные с сервера");
29       setStatus("Ошибка соединения");
30       console.error(err);
31     } finally {
32       setLoading(false);
33     }
34   }
35
36   useEffect(() => {
37     loadMessage();
38   }, []);
39
40   return (
41     <main className="page">
42       <section className="card">
43         <h1>React-клиент</h1>
44         <p className="status">{status}</p>
45
46         {loading && <p>Загрузка данных...</p>}
47         {error && <p className="error">{error}</p>}
48         {message && <p className="message">{message}</p>}
49
50         <button onClick={loadMessage}>Обновить данные</button>
51       </section>
52     </main>
53   );
54 }
55

```

Рисунок 46 – Пример написание файла App.jsx

В этом примере компонент отправляет запрос к серверу сразу после загрузки страницы. Также на странице есть кнопка, которая позволяет повторно выполнить запрос вручную.

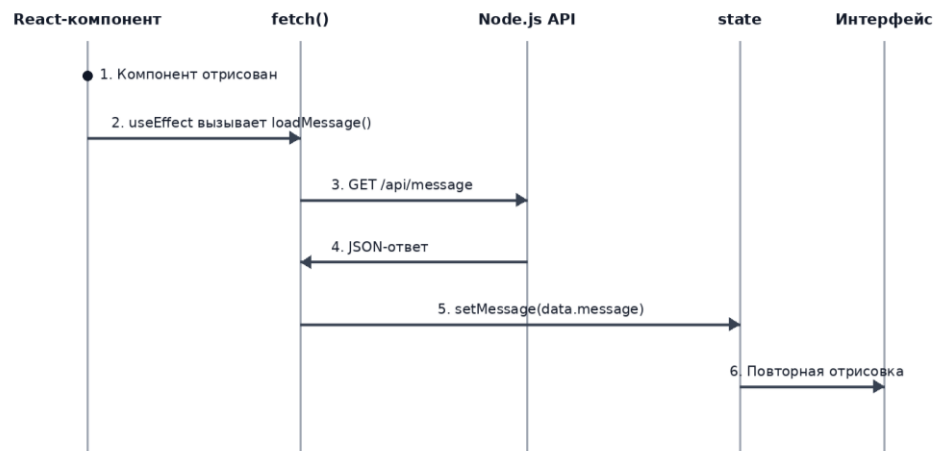


Рисунок 47 – Последовательность получения данных с сервера в React

6.9. Разбор кода получения данных

В начале файла подключаются хуки React:

```
import { useEffect, useState } from "react";
```

Рисунок 48 – Подключение хуков

Хук **useState** используется для хранения данных внутри компонента. В данном примере создаются четыре состояния:

- **message** - сообщение, полученное от сервера;
- **status** - текстовое описание текущего состояния запроса;
- **loading** - признак выполнения запроса;
- **error** - сообщение об ошибке, если запрос не выполнен.

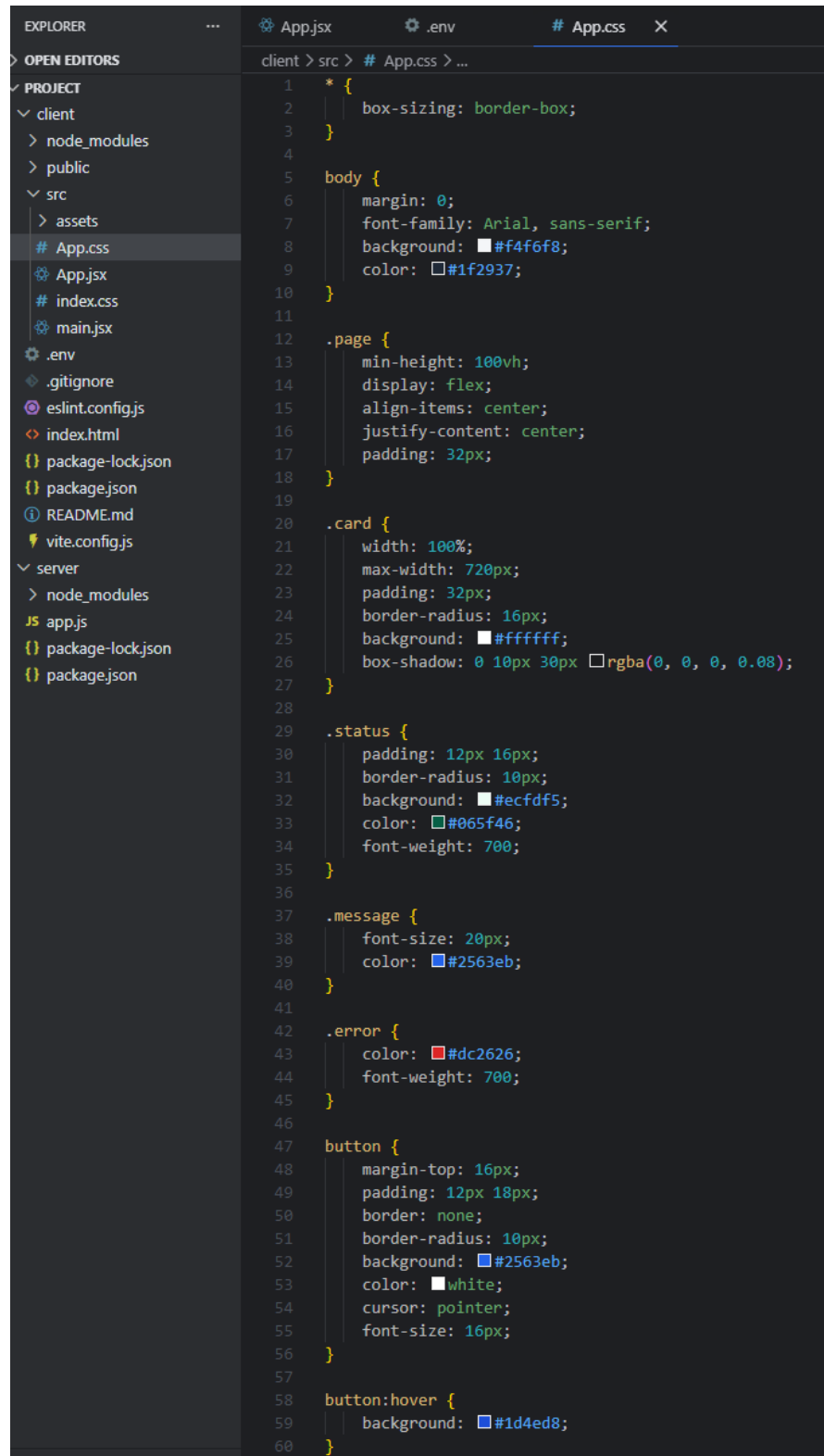
Функция **loadMessage()** выполняет основной запрос к серверу. Внутри неё используется **await fetch(...)**, то есть компонент ожидает ответ от сервера. После получения ответа вызывается метод **response.json()**, который преобразует тело ответа из JSON-строки в JavaScript-объект.

Если запрос прошёл успешно, значение **data.message** записывается в состояние **message**. После изменения состояния React автоматически обновляет интерфейс, и пользователь видит сообщение на странице.

Если сервер недоступен, указан неверный порт или маршрут **/api/message** отсутствует, блок **catch** перехватит ошибку. В этом случае пользователь увидит сообщение Не удалось получить данные с сервера, а подробная информация будет выведена в консоль браузера.

6.10. Добавление стилей клиентской части

Чтобы результат выглядел аккуратнее, добавим простые стили. Откройте файл `src/App.css` и замените его содержимое следующим кодом:

The image shows a code editor with a dark theme. On the left is the Explorer sidebar showing a file tree with folders like 'client', 'src', and 'server'. The 'App.css' file is selected. The main editor area shows the content of 'App.css' with line numbers from 1 to 60. The code defines styles for the root, body, .page, .card, .status, .message, .error, button, and button: hover states.

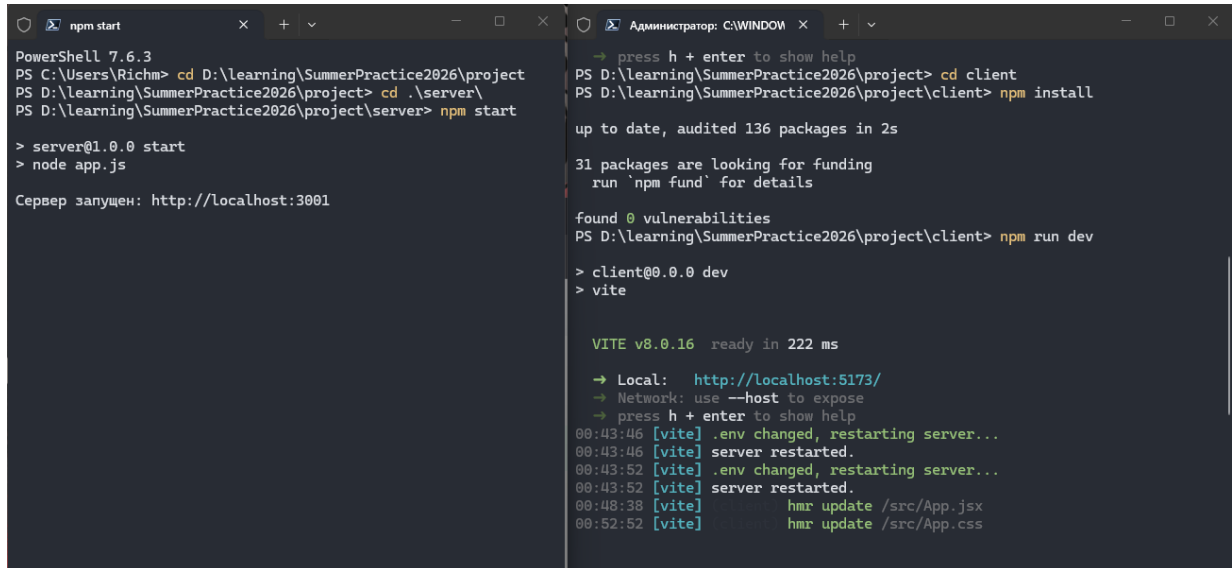
```
1  * {  
2    box-sizing: border-box;  
3  }  
4  
5  body {  
6    margin: 0;  
7    font-family: Arial, sans-serif;  
8    background: #f4f6f8;  
9    color: #1f2937;  
10  }  
11  
12  .page {  
13    min-height: 100vh;  
14    display: flex;  
15    align-items: center;  
16    justify-content: center;  
17    padding: 32px;  
18  }  
19  
20  .card {  
21    width: 100%;  
22    max-width: 720px;  
23    padding: 32px;  
24    border-radius: 16px;  
25    background: #ffffff;  
26    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.08);  
27  }  
28  
29  .status {  
30    padding: 12px 16px;  
31    border-radius: 10px;  
32    background: #ecfdf5;  
33    color: #065f46;  
34    font-weight: 700;  
35  }  
36  
37  .message {  
38    font-size: 20px;  
39    color: #2563eb;  
40  }  
41  
42  .error {  
43    color: #dc2626;  
44    font-weight: 700;  
45  }  
46  
47  button {  
48    margin-top: 16px;  
49    padding: 12px 18px;  
50    border: none;  
51    border-radius: 10px;  
52    background: #2563eb;  
53    color: white;  
54    cursor: pointer;  
55    font-size: 16px;  
56  }  
57  
58  button:hover {  
59    background: #1d4ed8;  
60  }
```

Рисунок 49 – Пример написания файла `App.css`

Данные стили не являются обязательными для работы клиент-серверного взаимодействия, но позволяют нагляднее увидеть результат работы приложения.

6.11. Проверка работы React-клиента

Для проверки необходимо, чтобы серверная и клиентская части были запущены одновременно. Откройте два терминала и запустите клиент и сервер:



```
PowerShell 7.6.3
PS C:\Users\Richm> cd D:\learning\SummerPractice2026\project
PS D:\learning\SummerPractice2026\project> cd .\server\
PS D:\learning\SummerPractice2026\project\server> npm start

> server@1.0.0 start
> node app.js

Сервер запущен: http://localhost:3001

Администратор: C:\WINDOW
→ press h + enter to show help
PS D:\learning\SummerPractice2026\project> cd client
PS D:\learning\SummerPractice2026\project\client> npm install

up to date, audited 136 packages in 2s

31 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS D:\learning\SummerPractice2026\project\client> npm run dev

> client@0.0.0 dev
> vite

VITE v8.0.16 ready in 222 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
00:43:46 [vite] .env changed, restarting server...
00:43:46 [vite] server restarted.
00:43:52 [vite] .env changed, restarting server...
00:43:52 [vite] server restarted.
00:48:38 [vite] hmr update /src/App.jsx
00:52:52 [vite] hmr update /src/App.css
```

Рисунок 50 – Запуск клиента и сервера в двух терминалах

После этого откройте в браузере адрес React-приложения:

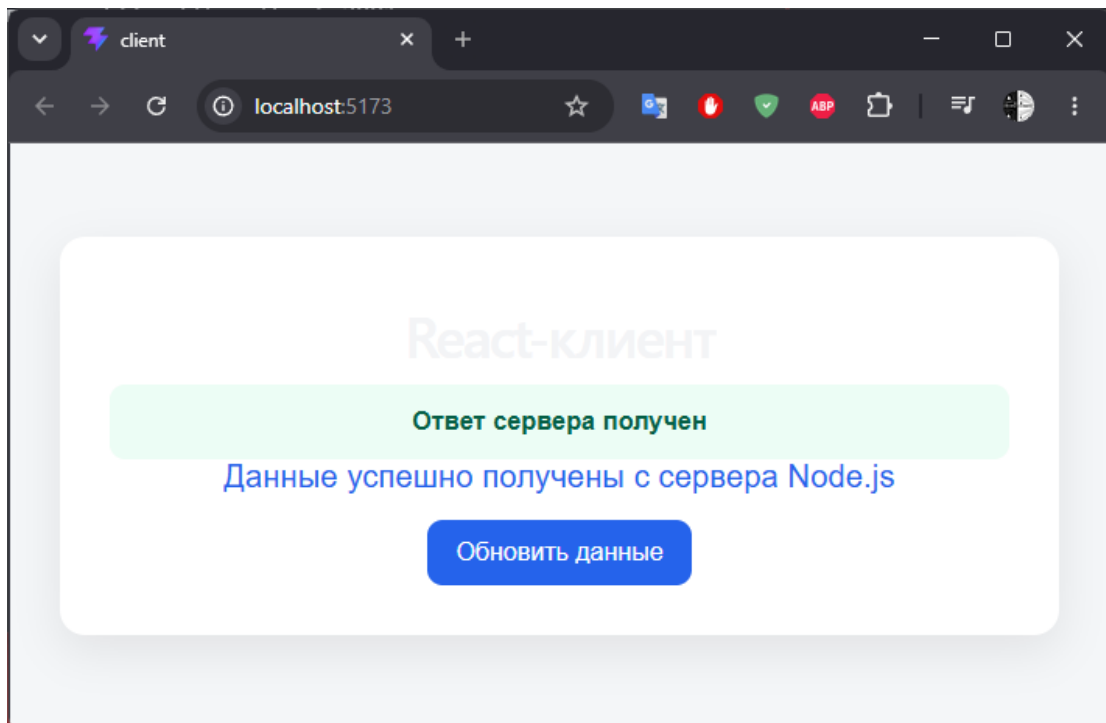


Рисунок 51 – Проверка работы клиента и сервера

Если сервер и клиент настроены правильно, на странице появится статус Ответ сервера получен и сообщение, возвращённое сервером.

6.12. Передача данных с React-клиента на сервер

Получение данных с сервера является только одной частью клиент-серверного взаимодействия. Во многих приложениях клиент также отправляет данные на сервер: например, данные формы регистрации, комментариев, заявку или сообщение обратной связи. Для этого обычно используется POST-запрос.

Для демонстрации добавим простой пример отправки текста с клиента на сервер. На сервере должен быть создан маршрут `/api/feedback`, который принимает JSON-объект.

6.13. Маршрут POST на сервере

В файл `server/app.js` нужно добавить следующий маршрут:

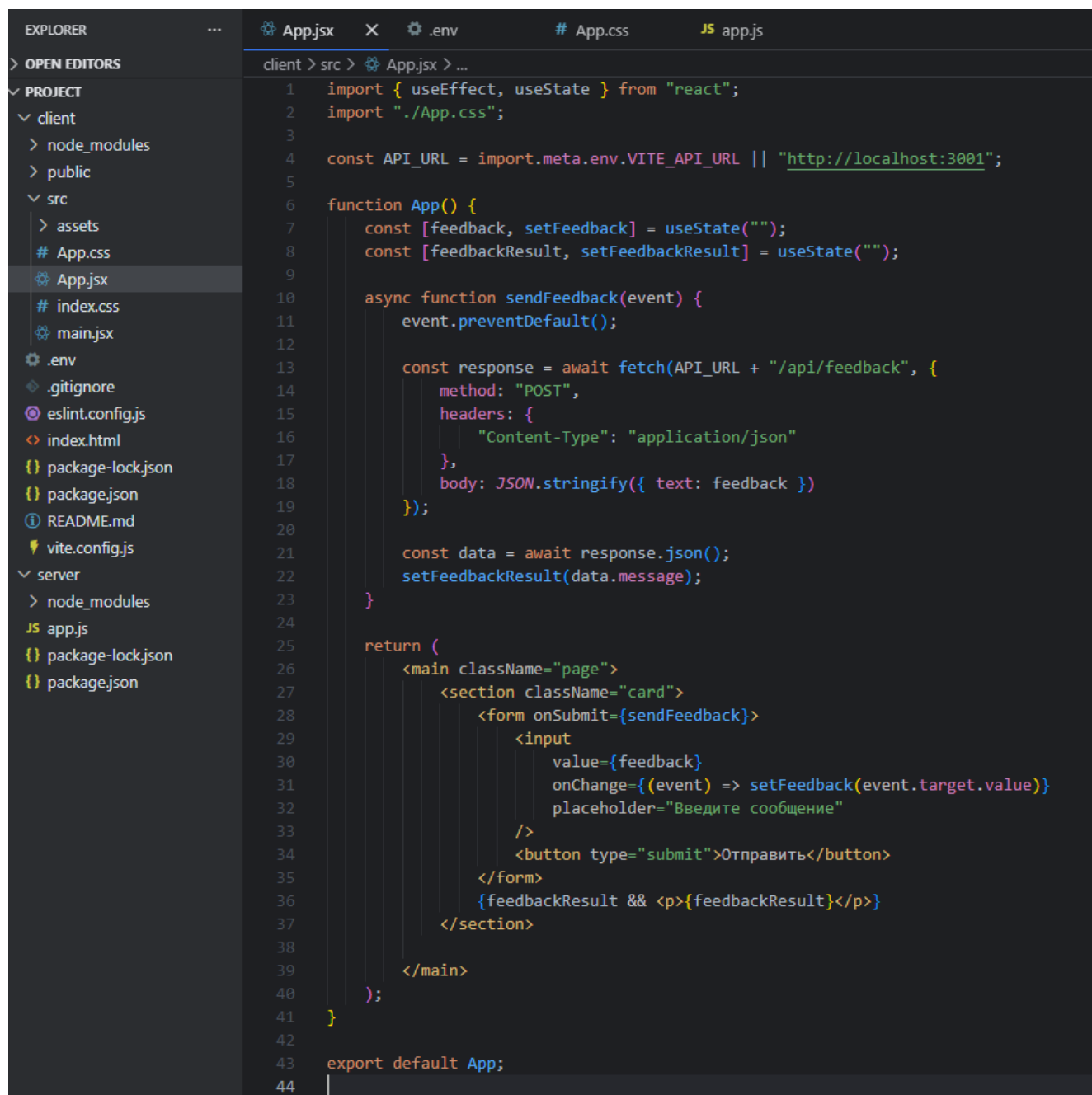
```
24 app.post("/api/feedback", function (request, response) {
25   const { text } = request.body;
26
27   if (!text || text.trim() === "") {
28     return response.status(400).json({
29       message: "Текст сообщения не должен быть пустым"
30     });
31   }
32
33   response.json({
34     message: "Сообщение успешно получено сервером",
35     receivedText: text
36   });
37 });
38
```

Рисунок 52 – Создание маршрута `/api/feedback` на серверной части

Этот маршрут получает поле `text` из тела запроса. Если поле пустое, сервер возвращает ошибку с кодом 400. Если данные заполнены, сервер возвращает подтверждение и текст, который был получен от клиента.

6.14. Форма отправки данных в React

Теперь в React-компонент можно добавить форму. Ниже приведён сокращённый пример функции отправки данных:



```
1  import { useEffect, useState } from "react";
2  import "../App.css";
3
4  const API_URL = import.meta.env.VITE_API_URL || "http://localhost:3001";
5
6  function App() {
7    const [feedback, setFeedback] = useState("");
8    const [feedbackResult, setFeedbackResult] = useState("");
9
10   async function sendFeedback(event) {
11     event.preventDefault();
12
13     const response = await fetch(API_URL + "/api/feedback", {
14       method: "POST",
15       headers: {
16         "Content-Type": "application/json"
17       },
18       body: JSON.stringify({ text: feedback })
19     });
20
21     const data = await response.json();
22     setFeedbackResult(data.message);
23   }
24
25   return (
26     <main className="page">
27       <section className="card">
28         <form onSubmit={sendFeedback}>
29           <input
30             value={feedback}
31             onChange={(event) => setFeedback(event.target.value)}
32             placeholder="Введите сообщение"
33           />
34           <button type="submit">Отправить</button>
35         </form>
36         {feedbackResult} && <p>{feedbackResult}</p>
37       </section>
38     </main>
39   );
40 }
41
42
43 export default App;
44
```

Рисунок 53 – Добавление формы в клиентскую часть

В данном коде используется **метод POST**. Заголовок Content-Type сообщает серверу, что данные отправляются в формате JSON. Метод **JSON.stringify()** преобразует JavaScript-объект в JSON-строку, которую можно отправить в теле запроса.

Такой пример показывает полный цикл обмена данными: пользователь вводит текст в интерфейсе, React отправляет его на сервер, сервер обрабатывает запрос и возвращает ответ.

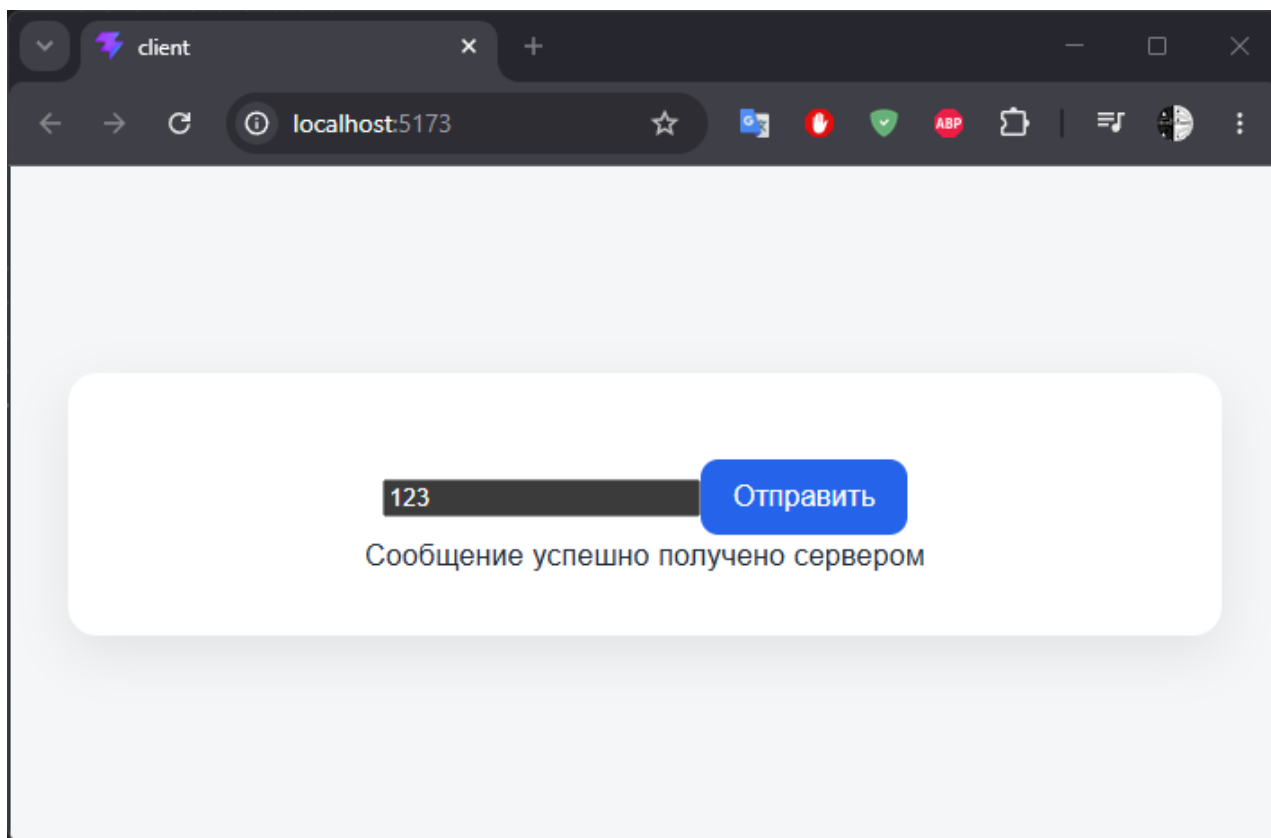


Рисунок 54 – Проверка работы формы в браузере

7. Типичные ошибки и способы исправления

При первом создании клиент-серверного приложения ошибки возникают часто. Это нормально. Главное - проверять проблему последовательно: сначала убедиться, что сервер запущен, затем проверить адрес API, после этого посмотреть консоль браузера и терминал.

Ошибка или ситуация	Возможная причина	Что сделать
npm не является внутренней или внешней командой	Node.js не установлен или терминал открыт до установки Node.js.	Переустановить Node.js или перезапустить PowerShell / VS Code.
Cannot find module express	Пакет express не установлен в папке server.	Перейти в папку server и выполнить <code>npm install express cors</code> .
Port 3001 is already in use	Порт уже занят другим процессом или сервер запущен повторно.	Остановить старый сервер <code>Ctrl + C</code> или изменить номер порта.
Failed to fetch	React-клиент не может получить ответ от сервера.	Проверить, запущен ли сервер, и открыть <code>http://localhost:3001/api/message</code> в браузере.
CORS error	Сервер не разрешает запросы с другого порта.	Проверить наличие <code>app.use(cors())</code> в <code>app.js</code> .
Пустая страница React	Ошибка в компоненте или неверный импорт файла.	Открыть инструменты разработчика в браузере и посмотреть Console.
Cannot GET /api/message	На сервере нет маршрута с таким адресом или он написан с ошибкой.	Сравнить адрес в <code>fetch</code> и маршрут <code>app.get</code> на сервере.
npm run dev не запускается	Команда выполняется не в папке client или зависимости не установлены.	Перейти в папку client и выполнить <code>npm install</code> , затем <code>npm run dev</code> .

Практическое правило: если не работает React-страница с данными, сначала нужно проверить сервер отдельно. Для этого откройте в браузере адрес `http://localhost:3001/api/message`. Если JSON отображается, сервер работает, и ошибку следует искать на стороне клиента.

Для поиска ошибок удобно использовать инструменты разработчика браузера. В большинстве браузеров они открываются клавишей F12 или сочетанием `Ctrl + Shift + I`. Вкладка Console показывает ошибки JavaScript и CORS, а вкладка Network позволяет увидеть отправленные HTTP-запросы, их статусы и ответы сервера.

Что проверить	Где смотреть	Что означает
Статус 200	Network	Запрос выполнен успешно, сервер вернул ответ.
Статус 404	Network	Адрес маршрута указан неверно или такого маршрута нет на сервере.
Статус 500	Network / терминал сервера	На сервере возникла ошибка при обработке запроса.
CORS error	Console	Клиент и сервер работают на разных портах, но сервер не разрешил запрос.
Failed to fetch	Console	Сервер не запущен, указан неверный порт или адрес API.

8. Рекомендуемые источники и справочные материалы

При выполнении практической работы студентам рекомендуется пользоваться официальной документацией и учебными справочниками. Ниже приведены источники, которые можно использовать для уточнения синтаксиса JavaScript, React, Node.js, Express, Vite, npm и HTTP-запросов.

№	Источник	Для чего использовать
1	METANIT. Раздел веб-разработки. https://metanit.com/web/	Общий справочник по JavaScript, HTML, CSS, Node.js, React и другим веб-технологиям.
2	Node.js. Официальный сайт и документация. https://nodejs.org/en	Проверка назначения Node.js, установка, npm и справочные материалы по среде выполнения.
3	React. Официальная документация. https://react.dev/learn	Изучение компонентов, состояния, эффектов и современной логики React.
4	React. Build a React app from Scratch. https://react.dev/learn/build-a-react-app-from-scratch	Пояснение, зачем нужен build tool, например Vite, при создании React-приложения.
5	Vite. Официальное руководство. https://vite.dev/guide/	Создание и запуск проекта Vite, команды разработки и сборки.
6	Express. Routing guide. https://expressjs.com/en/guide/routing/	Создание маршрутов app.get(), app.post() и обработка адресов API.
7	MDN Web Docs. Overview of HTTP. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview	Понимание HTTP, запросов, ответов и клиент-серверного обмена.
8	MDN Web Docs. Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API	Работа с fetch() на стороне клиента.
9	npm Docs. package.json. https://docs.npmjs.com/files/package.json/	Понимание файла package.json, зависимостей и команд запуска.